



Operating System

Lab 12

Fasiha Tariq – 53289

Submitted to:

Ma'am Ayesha Ikram

Date: November 3rd, 2025

TASKS

1.

[illegible]

2.

```
student@student-virtual-machine:~$ pico signal.c
student@student-virtual-machine:~$ cat signal.c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void catcher(int sigtype)
{
    printf("--- I got the signal\n");
    signal(SIGINT, catcher);
}

int main(void)
{
    int i;
    signal(SIGINT, catcher);
    for (i = 0; i < 20; i++)
    {
        printf("working...\n");
        sleep(1);
    }
}
student@student-virtual-machine:~$ gcc signal.c -o signal
student@student-virtual-machine:~$ ./signal
working...
working...
working...
working...
working...
working...
working...
^C--- I got the signal
working...
working...
working...
working...
working...
working...
--- I got the signal
working...
^Cworking...
working...
working...
working...
working...
working...
```

3.

```
student@student-virtual-machine:~$ pico signals.c
student@student-virtual-machine:~$ cat signals.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void signal_catcher(int sig)
{
    printf("Caught signal %d\n", sig);
}

int main(void)
{
    #ignore Ctrl-C
    printf("=== SIGNAL HANDLER DEMO ===\n");
    printf("Process ID (PID): %d\n\n", getpid());
    printf("You can send signals using:\n");
    printf("  - Ctrl+C (SIGINT)\n");
    printf("  - kill -SIGTERM %d (SIGTERM)\n", getpid());
    printf("  - kill -SIGINT %d (SIGINT)\n\n", getpid());
    printf("Press Ctrl+C or use kill commands while this runs...\n\n");

    int i;

    signal(SIGINT, SIG_IGN);
    printf("1. SIGINT (Ctrl+C) is now being IGNORED.\n");
    for (i = 0; i < 5; i++)
    {
        printf("  [Ignoring Phase] Working... (%d)\n", i + 1);
        sleep(1);
    }
    #Ctrl-C action working
    signal(SIGINT, SIG_DFL);
    printf("\n2. SIGINT is now set to DEFAULT action.\n");
    printf("  Press Ctrl+C now to terminate, or wait to continue...\n");
    for (i = 0; i < 5; i++)
    {
        printf("  [Default Phase] Working... (%d)\n", i + 1);
        sleep(1);
    }
    #Custom Handler
    signal(SIGTERM, signal_catcher);
    printf("\n3. Custom handler for SIGTERM is now active.\n");
    printf("  Run: kill -SIGTERM %d\n", getpid());
    printf("  The program will catch it and continue running.\n");
    for (i = 0; i < 10; i++)
    {
        printf("  [Custom Handler Phase] Working... (%d)\n", i + 1);
        sleep(1);
    }

    printf("\n=== Program finished ===\n");
    return 0;
}
```

```
student@student-virtual-machine:~$ gcc signals.c -o signals
student@student-virtual-machine:~$ ./signals
=== SIGNAL HANDLER DEMO ===
Process ID (PID): 158429

You can send signals using:
- Ctrl+C (SIGINT)
- kill -SIGTERM 158429 (SIGTERM)
- kill -SIGINT 158429 (SIGINT)

Press Ctrl+C or use kill commands while this runs...

1. SIGINT (Ctrl+C) is now being IGNORED.
  [Ignoring Phase] Working... (1)
  [Ignoring Phase] Working... (2)
  [Ignoring Phase] Working... (3)
^C [Ignoring Phase] Working... (4)
  [Ignoring Phase] Working... (5)

2. SIGINT is now set to DEFAULT action.
  Press Ctrl+C now to terminate, or wait to continue...
  [Default Phase] Working... (1)
  [Default Phase] Working... (2)
^C
```

```
3. Custom handler for SIGTERM is now active.
  Run: kill -SIGTERM 158433
  The program will catch it and continue running.
  [Custom Handler Phase] Working... (1)
  [Custom Handler Phase] Working... (2)
  [Custom Handler Phase] Working... (3)
  [Custom Handler Phase] Working... (4)
  [Custom Handler Phase] Working... (5)
  [Custom Handler Phase] Working... (6)
  [Custom Handler Phase] Working... (7)
  [Custom Handler Phase] Working... (8)
  [Custom Handler Phase] Working... (9)
  [Custom Handler Phase] Working... (10)

=== Program finished ===
```

4.

```
student@student-virtual-machine:~$ pico sig.c
student@student-virtual-machine:~$ cat sig.c
#include <stdio.h>
#include <signal.h>

int main()
{
    signal(SIGINT, SIG_IGN);
    while(1)
        printf("You can't kill me with SIGINT anymore\n");
    return 0;
}
student@student-virtual-machine:~$ gcc sig.c -o sig
student@student-virtual-machine:~$ ./sig
```

[illegible]

5. a)

```
student@student-virtual-machine:~$ pico sigint.c
student@student-virtual-machine:~$ cat sigint.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void handle_sigint(int sig) {
    printf("\nCaught signal %d, coming out...\n", sig);
    exit(0);
}

int main() {
    signal(SIGINT, handle_sigint);

    while (1) {
        printf("Going to sleep for a second...\n");
        sleep(1);
    }

    return 0;
}
student@student-virtual-machine:~$ gcc sigint.c -o sigint
student@student-virtual-machine:~$ ./sigint
Going to sleep for a second...
Going to sleep for a second...
Going to sleep for a second...
Going to sleep for a second...
Going to sleep for a second...
Going to sleep for a second...
^C
Caught signal 2, coming out...
```

b)

```
student@student-virtual-machine:~$ pico sigstp.c
student@student-virtual-machine:~$ cat sigstp.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void handle_signal(int sig) {
    printf("The interrupt signal is (%d).\n", sig);
    exit(0);
}

int main() {
    signal(SIGSTP, handle_signal); // usually signal number 22

    while (1) {
        printf("Hello GeeksforGeeks...\n");
        sleep(1);
    }

    return 0;
}

Hello GeeksforGeeks...
Hello GeeksforGeeks...
Hello GeeksforGeeks...
Hello GeeksforGeeks...
Hello GeeksforGeeks...
^ZThe interrupt signal is (20).
```